



NOWITNESS

Security Audit Report

Lace Browser Extension & API

Date November 21, 2025
Project Lace Midnight Preview
Version 2.34.0

Contents

Disclosure	1
Disclaimer	2
Scope	3
Assessment Overview	4
Assessment Components	5
Executive Summary	6
Code Base	7
Severity Classification	8
Finding Severity Ratings	9
Findings	10
[C 501] Unshielded Wallet Private Key Memory Leak	12
[M 301] Weak mitigations against XSS from malicious token metadata	14
[M 302] PBKDF2 Insufficient Iterations (EMIP-3 Encryption)	16
[M 303] Critical dependencies use floating semver ranges	19
[M 304] Extension exposes its bundles to every origin	20
[M 305] Midnight unlock password and decrypted ZSwap key are never zeroized ..	22
[L 201] Unlimited Password Attempts on Wallet Unlock	23
[L 202] Password Policy Allows Weak Passwords	24
[L 203] Feature-flag cache is controllable via page localStorage	26
[L 204] Wallets never auto-lock after unlock	27
[L 205] Missing CDDL/Bech32 Address Validation	28
[I 101] Unsafe Onboarding Flows	30
[I 102] Missing <code>Contacts / Address Book</code> Feature	33
[I 103] Unimplemented Wallet API Codecs	34
[I 104] Suggestion: TOR Network Anonymity as Optional Privacy Enhancement ..	36

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **IO | Global** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this penetration test was limited to the Lace browser extension as deployed in the provided test environment and did not include the underlying blockchain, backend services, host browser or operating system, or any external dApps or websites.

Scope

In-Scope Components

- Browser Extension (popup, background scripts, content scripts)
- Web Wallet Interface (if applicable)
- APIs and Network Communication
- Storage (localStorage, chrome.storage, IndexedDB)
- Authentication and Session Handling
- Password / Seed-phrase Handling Modules
- User Onboarding and Educational Content
- Domains and Subdomains
- IP Addresses

Out-of-Scope Components

- Third-party dApps, unless whitelisted for integration tests
- Remote and local proof server
- Remote node RPC server
- Remote indexer

Assessment Overview

From **October 27, 2025** to **November 21, 2025**, **IO | Global** engaged No Witness Labs to conduct a comprehensive security assessment of the **Lace Midnight Preview** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

Assessment Components

Manual review

Our manual review covers the Lace Midnight browser extension, associated web UI, and supporting APIs. It focuses on a wide range of attack surfaces and failure modes defined in the agreed test plan, including but not limited to:

- Web-facing security (XSS, CSRF, origin checks, malicious content)
- Extension manifest scope, host permissions, and content-script scoping
- Storage security for mnemonics, private keys, and passwords (EMIP-3 alignment)
- Authentication and session management (unlock flows, timeouts, persistence)
- Password strength and offline-guess resistance
- Seed phrase generation, display, and backup confirmation UX
- Transaction preview versus signed-transaction integrity
- Message passing between in-page, content, and background scripts
- Network/API security (remote node/indexer usage, TLS/MITM resilience)
- Token spam and phishing protections (unsafe dApp requests, “Hide balance” usage)
- Telemetry, logging, and analytics data minimization
- New user onboarding, safety prompts, and user education

Executive Summary

Project Overview

Midnight is a data-protection blockchain that uses zero-knowledge proofs to safeguard sensitive commercial and personal data and is designed to operate as a privacy-focused sidechain to Cardano.

Lace is a non-custodial light wallet platform. For Midnight, it is provided as the Lace Midnight Preview wallet, a browser-extension wallet for the Midnight network that runs as a Chrome-compatible extension and is used to create a Midnight wallet, set a spending password, and back up the recovery phrase before interacting with the Midnight testnet.

This engagement assessed the client-side security of the Lace Midnight Preview extension and its associated web flows. The focus was on password and lock/unlock behaviour, seed generation and backup UX, transaction review and signing, token and phishing exposure, and related telemetry and storage behaviour. The underlying Midnight protocol, Lace back-end services, and third-party dApps were treated as external dependencies and remained out of scope.

Code Base

Repository

<https://github.com/input-output-hk/lace-platform.git>

Tag

midnight-extension@2.34.0

Commit

80ae8396dcc33cb1854cd7d46559dc4f7402e4ee

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Status
	5	Critical	1
	4	High	0
	3	Medium	5
	2	Low	5
	1	Informational	4

Findings

ID	Title	Severity	Status
501	Unshielded Wallet Private Key Memory Leak	Critical	Acknowledged
301	Weak mitigations against XSS from malicious token metadata	Medium	Acknowledged
302	PBKDF2 Insufficient Iterations (EMIP-3 Encryption)	Medium	Acknowledged
303	Critical dependencies use floating semver ranges	Medium	Acknowledged
304	Extension exposes its bundles to every origin	Medium	Acknowledged
305	Midnight unlock password and decrypted ZSwap key are never zeroized	Medium	Acknowledged
201	Unlimited Password Attempts on Wallet Unlock	Low	Acknowledged
202	Password Policy Allows Weak Passwords	Low	Acknowledged
203	Feature-flag cache is controllable via page local-Storage	Low	Acknowledged
204	Wallets never auto-lock after unlock	Low	Acknowledged
205	Missing CDDL/Bech32 Address Validation	Low	Acknowledged
101	Unsafe Onboarding Flows	Informational	Communicated
102	Missing <code>Contacts / Address Book</code> Feature	Informational	Communicated
103	Unimplemented Wallet API Codecs	Informational	Communicated

ID	Title	Severity	Status
104	Suggestion: TOR Network Anonymity as Optional Privacy Enhancement	Informational	Communicated

[C 501] Unshielded Wallet Private Key Memory Leak

	Level	Severity	Status
	5	Critical	Acknowledged

Category: Extension & Storage Security

Affected File: `packages/module/blockchain-midnight/src/store/dependencies.ts`

Affected Components: Midnight unshielded wallet initialization, private key cleanup

Description

The `nightExternalKeyBuffer` (unshielded wallet private key) is not erased from memory after wallet initialization. Cleanup code is commented out due to bug LW-13756: "Sending unshielded tokens fails if this key gets erased."

Private key cleanup explicitly disabled

- Observed: `packages/module/blockchain-midnight/src/store/dependencies.ts:135-136` contains `// nightExternalKeyBuffer.fill(0); // COMMENTED OUT!`
- Why problematic: Raw 32-byte private key remains in browser heap memory indefinitely. While JavaScript heap is garbage collected non-deterministically, the key remains accessible until collection occurs. Exploitation requires either (1) malicious browser extension with debugger/memory access permissions, (2) physical device access for memory forensics, or (3) browser crash dumps which may persist on disk. This deviates from standard cryptographic key handling practices and defense-in-depth approaches.

Code evidence:

```

1 // Line 104-109: Decrypt private key
2 const nightExternalKeyBuffer = ByteArray(
3   await emip3decrypt(ByteArray.fromHex(nightExternalKey.encryptedKey), password)
4 );
5
6 // Lines 118-133: Use key to build wallet
7 unshieldedWallet = await WalletBuilder.build({ seed: nightExternalKeyBuffer });
8
9 // Lines 135-136: Cleanup DISABLED
10 // TODO: LW-13756 Sending unshielded tokens fails if this key gets erased
11 // nightExternalKeyBuffer.fill(0); // COMMENTED OUT!
12
13 return { walletFacade: new WalletFacade(shieldedWallet, unshieldedWallet) };
14 // Private key remains in memory!
15
16 // Compare: Shielded wallet (line 102) - WORKS CORRECTLY
  
```

```
17 zswapKeyBuffer.fill(0); // Key properly erased
```

Recommendation

Fix the root cause of bug LW-13756 where the WalletBuilder retains a reference to the seed buffer instead of creating a defensive copy. Once fixed, uncomment the `nightExternalKeyBuffer.fill(0)` cleanup code at line 135-136 to enable proper key erasure. Verify that unshielded token transfers continue to work correctly after enabling key erasure, and add automated regression tests to prevent future breakage. The shielded wallet already implements this pattern correctly (line 102), demonstrating that secure key cleanup is achievable without breaking functionality.

[M 301] Weak mitigations against XSS from malicious token metadata

	Level	Severity	Status
	3	Medium	Acknowledged

Category: Web-Facing Security

Description

Dynamic analysis shows the Lace and Midnight extension UIs render token and NFT metadata images exactly as supplied by the underlying blockchain APIs. Whatever string appears in the asset's `image` field is placed into `<img>` tags inside the popup and full-page experiences. The manifest's CSP allows `img-src * data:` to keep decentralized metadata working, but as a result any remote URL or `data:` URI provided by an attacker-controlled asset is displayed without sanitisation, proxying, or scheme restrictions. Because token metadata is entirely user-controlled, the UI effectively relies on Chromium/Firefox/Safari heuristics to prevent script execution rather than enforcing its own guardrails.

Testing confirmed that minting or importing a token with a `data:image/svg+xml` payload containing `<foreignObject onload=...>` results in that SVG being persisted and rendered across the portfolio views. Modern browsers currently neutralize script when SVGs are used via `<img>`, so the payload did not fire during this assessment. Reproduction therefore hinges on implementation or platform changes rather than a deterministic PoC. Any UI change that inlines SVG markup, opens the metadata URL in a richer viewer, uses `<object>` / `<iframe>` for previews, or runs on a browser lacking the latest SVG hardening would immediately convert those stored payloads into active XSS within the extension. Because the CSP explicitly permits `data:` and wildcard hosts, there is no additional containment—maintaining a permissive CSP to support decentralized metadata makes it even more critical not to depend solely on browser-level mitigations.

Those heuristics (Chromium's "SVG-as-image" sandbox, Firefox's `SVGImageElement` restrictions, Safari's WebKit fix for scriptable image documents) are not guaranteed forever. Edge cases that would reintroduce the bug include: a future feature that displays rich NFT previews by inlining the SVG markup, any fallback that opens the metadata URL in a new tab/window, platforms/browsers that lag behind the mitigation, or accidental reuse of the same metadata inside a DOM sink such as `dangerouslySetInnerHTML`. If that happens, a script running inside the extension could, in theory, read and exfiltrate sensitive state already stored in memory (authorised dApp lists, transaction drafts, decrypted wallet context, clipboard contents, encrypted recovery phrases, and encrypted passwords), making this a direct confidentiality risk rather than a cosmetic bug.

Recommendation

- Treat token/NFT metadata as untrusted input. Validate and sanitize all metadata fields before use and strictly reject any mechanisms that would allow for script execution.

- Apply the same validation, sanitization, and scheme restrictions to any component that accepts dApp-provided icons (`ProfilePicture.*`, gallery views, recent activity), so future UI tweaks cannot silently reintroduce script-capable renderers.

[M 302] PBKDF2 Insufficient Iterations (EMIP-3 Encryption)

	Level	Severity	Status
	3	Medium	Acknowledged

Category: Extension & Storage Security

Affected Files:

- `packages/module/blockchain-midnight/src/exported-modules/in-memory-wallet-integration/in-memory-wallet-integration-factory.ts`
- `cardano-js-sdk/packages/key-management/src/emip3.ts`

Affected Components: Midnight wallet encryption, password-based key derivation

Description

Lace Midnight wallet uses PBKDF2-HMAC-SHA512 with 19,162 iterations to derive encryption keys from user passwords (via EMIP-3 specification). This is approximately 31x below OWASP 2023 recommended standards (600,000 iterations minimum), which may expose user passwords to offline brute-force attacks.

Low iteration count enables password brute-forcing

- Observed: [@cardano-sdk/key-management](#) EMIP-3 implementation uses only 19,162 iterations.
- Usage in midnight-extension:
 1. Extension loads `vaultInMemory` module:
 - `apps/midnight-extension/src/util/all-modules.ts:25,54`
 2. User creates/restores wallet via:
 - `packages/module/vault-in-memory/src/onboarding-options.tsx`
 3. Calls `createInMemoryWalletEntityFactory` in:
 - `packages/module/vault-in-memory/src/util.ts:37-42` → encrypts recovery phrase
 4. Calls `initializeWallet` in:
 - `packages/module/blockchain-midnight/src/exported-modules/in-memory-wallet-integration/in-memory-wallet-integration-factory.ts:127`
 5. Line 43: `await emip3encrypt(keyHex, password)` - encrypts private keys (Zswap, NightExternal)
 6. Line 119: `await emip3encrypt(seed, password)` - encrypts seed
- Why problematic:
 - PBKDF2 derives encryption key from user password using 19,162 iterations.
 - An attacker who obtains encrypted wallet data (via malware, backup theft, or storage breach) can test millions of password candidates per second using GPU/ASIC clusters.
 - With approximately 31x fewer iterations than OWASP 2023 recommended standards, weak passwords may be cracked in hours instead of years.
 - Once password is recovered, attacker calls `emip3decrypt` to recover all private keys and seed.

Attack flow

1. Attacker obtains encrypted keys/seed from storage (malware, backup theft)
2. Attacker brute-forces password: `PBKDF2(candidate, salt, 19162)` → test decryption
3. Once password found: `emip3decrypt(encryptedData, password)` → recover private keys
4. Attacker drains wallet funds and impersonates user

Code evidence:

```

1 // packages/module/blockchain-midnight/src/exported-modules/in-memory-
  wallet-integration/in-memory-wallet-integration-factory.ts typescript
2
3 // Line 43: Encrypt private keys with password
4 const keyHex = ByteArray(derivationResult.key);
5 return HexBytes.fromByteArray(
6   ByteArray(await emip3encrypt(keyHex, password)), // Password protects key
7 );
8
9 // Line 119: Encrypt seed with password
10 const encryptedSeed = HexBytes.fromByteArray(
11   ByteArray(await emip3encrypt(seed, password)), // Password protects seed
12 );
13
14 // EMIP-3 implementation (cardano-js-sdk)
15 const PBKDF2_ITERATIONS = 19_162; // ~31x below OWASP 2023 recommended minimum
16
17 const derivedKey = await crypto.subtle.deriveBits(
18   {
19     name: 'PBKDF2',
20     hash: 'SHA-512',
21     salt,
22     iterations: PBKDF2_ITERATIONS // Only 19,162 iterations
23   },
24   passwordKey,
25   32 * 8
26 );

```

Recommendation

1. Increase PBKDF2 iterations to meet current security standards
 - Update `PBKDF2_ITERATIONS` to 600,000 iterations (OWASP 2023 recommended minimum for PBKDF2-HMAC-SHA512) in cardano-js-sdk EMIP-3 implementation.

- Benchmark performance impact on target devices (browser extensions, mobile) to ensure acceptable user experience.
 - Coordinate with Cardano SDK maintainers for ecosystem-wide implementation.
2. Consider migration to Argon2id (memory-hard KDF)
- Argon2id provides resistance against GPU/ASIC attacks through memory-hard properties (recommended: 64MB memory, 3 iterations, parallelism=4).
 - Evaluate compatibility with existing EMIP-3 encrypted data and migration path.

[M 303] Critical dependencies use floating semver ranges

	Level	Severity	Status
	3	Medium	Acknowledged

Category: Code Quality & Communication Security

Description:

Package installs do not produce deterministic builds because most first-party dependencies are declared with caret/tilde ranges. Running `npm install` on a clean checkout (or adding any new workspace) immediately bumps packages such as `@sentry/react`, `redux`, `rxjs`, `typescript-eslint`, `redux-persist`, and numerous Nx/React Native toolchains to whatever patch/minor release happens to be latest at install time. The workspace does ship a `package-lock.json`, but regenerating it is part of the documented workflow (`npm install` is the default in scripts and CI), so any developer, CI runner, or malicious contributor can introduce a new transitive payload simply by reinstalling. Because Lace ships browser extensions and mobile apps that must withstand supply-chain attacks, silently accepting unreviewed dependency updates undermines the stated requirement for deterministic, auditable builds.

Recommendation:

- Replace all caret/tilde ranges in `dependencies` and `devDependencies` with exact versions, and enforce the policy via `npmPkgJsonLint`/CI so future diffs fail if floating ranges are introduced.
- Switch all install steps (developer onboarding, CI, release builds) to `npm ci` so the committed lockfile is always honored, and document that regenerating the lockfile requires security review.
- Consider a “frozen lockfile” approach (e.g., `npm shrinkwrap` or Renovate with manual approval) so dependency bumps happen only through deliberate, trackable PRs that can be audited for security impact.

[M 304] Extension exposes its bundles to every origin

	Level	Severity	Status
	3	Medium	Acknowledged

Category: Web-Facing Security

Description

The Lace and Midnight manifests declare `host_permissions: ["<all_urls>"]` and inject both `isolated-script.js` and `injected-script.js` into every HTTP(S) and `file://` origin.

This means that any site can immediately interact with the wallet bridge—even though Lace does not explicitly expose a `window.cardano` object on every page.

In practice, a hostile origin can still post `feature-flags#getFeatureFlags` messages, call Midnight wallet APIs, and access the global `window.midnight` object that the injected bundle registers.

Because the scripts run in the page's MAIN world, `window.midnight` becomes reachable anywhere Lace runs, contradicting Midnight's privacy goals and confirming that arbitrary origins obtain wallet-level reachability solely due to the blanket `<all_urls>` scope. Example response from `feature-flags#getFeatureFlags`:

```

1  {
2    "baseChannelName": "feature-flags",
3    "messageId": "73230f4d-b17d-45ba-99e7-af77629091c4",
4    "response": [
5      {
6        "key": "BLOCKCHAIN_MIDNIGHT_DAPP_CONNECTOR"
7      },
8      {
9        "key": "SEND_FLOW"
10     },
11     {
12       "key": "ANALYTICS_POSTHOG"
13     },
14     {
15       "key": "BLOCKCHAIN_MIDNIGHT"
16     },
17     {
18       "key": "BLOCKCHAIN_MIDNIGHT_REMOTE_PROOF_SERVER",
19       "payload": {
20         "2": "https://lace-dev.proof-pub.stg.midnight.tools",
21         "3": "https://lace-dev.proof-pub.stg.midnight.tools",
22         "url": "https://lace-dev.proof-pub.stg.midnight.tools"

```

```
23     }
24   },
25   {
26     "key": "FEATURES_POSTHOG"
27   }
28 ]
29 }
```

The manifest also exposes every bundled JS/WASM (and Expo dev bundles) via `web_accessible_resources` that match `<all_urls>`. Any website can request `chrome-extension://<id>/js/isolated-script.js`, which enables easy user fingerprinting and allows hostile pages to reuse privileged helper code. These settings grant visibility into local HTML files, unnecessarily increasing the wallet's attack surface.

Reproduction requires no special setup. Install the extension, browse to any origin, and observe `chrome-extension://<id>/js/isolated-script.js` loading automatically in DevTools. Executing `window.postMessage({ messageId: crypto.randomUUID(), request: { method: 'midnight-authenticator#requestAccess', args: [] } }, window.origin);` from that page opens the Lace permission prompt even though the origin was never allowed, illustrating that hostile sites control when users see authorization dialogs.

Midnight is primarily a privacy-focused blockchain and the extension's current behavior directly undermines those privacy guarantees. Blanket content-script injection and making bundled assets reachable from any origin enable easy cross-site fingerprinting, allow hostile origins to detect and correlate wallet state or user interactions across sites, and create avenues for coercive or tricked permission prompts that can link browsing activity to on-chain identities. Exposing local files and development bundles further increases metadata leakage and the risk of deanonymization. Together, these effects increase linkability, reduce anonymity sets, and harm Midnight's core privacy objectives.

Recommendation

- Do not inject content scripts (`isolated-script.js`, `injected-script.js`) on every origin by default. Only inject these necessary scripts after the wallet is connected, and then any remaining needed scripts after the current dApp has been added to the "Authorized dApps" whitelist by the user.
- Do not include `file:/**/*` or `all_frames` unless there is a clear, justified use case. Loading scripts on local files or all iframes is not standard and unnecessarily increases attack surface.
- Only expose individual, necessary JS/WASM assets via `web_accessible_resources` to trusted and user-approved origins. Do not expose entire directories or development bundles to all websites.

[M 305] Midnight unlock password and decrypted ZSwap key are never zeroized

	Level	Severity	Status
	3	Medium	Acknowledged

Category: Extension And Storage Security

Affected Components: Midnight unlock flow, password handling, ZSwap key lifecycle

Description

Runtime testing of the Midnight unlock flow shows that, after the user supplies their wallet password, both the plaintext password bytes and the decrypted ZSwap key remain resident in the Lace extension process for as long as that session survives. Heap snapshots taken from Chrome DevTools immediately after a successful unlock continue to reveal the password `Uint8Array` and the derived ZSwap key objects, and repeating the snapshots minutes later (without any active Midnight activity) shows the same buffers still allocated. There is no observable zeroization or reallocation step once the wallet finishes building.

This creates a prolonged exposure window: any attacker with debugging access, crash-dump collection, or a malicious companion extension could attach to the Lace process and read those arrays directly, bypassing the need to brute-force the encrypted wallet storage. Because the extension already injects into every origin, a compromised site could attempt heap-spray or DevTools-based attacks to capture the lingering key material. The behavior also violates Lace's own storage-security goal of zeroizing secrets immediately after use.

Recommendation

- Immediately overwrite (e.g., `fill(0)` or platform-appropriate zeroization) the password buffer and decrypted ZSwap key as soon as the Midnight wallet builder finishes using them, and ensure no additional references remain in closures or Redux state.
- Prefer short-lived scoped buffers or WebCrypto `CryptoKey` objects for intermediate secrets so garbage collection can reclaim them quickly.
- Add regression tests or instrumentation that fails builds when heap snapshots still contain sensitive Midnight secrets after the unlock routine returns.

[L 201] Unlimited Password Attempts on Wallet Unlock

	Level	Severity	Status
	2	Low	Acknowledged

Category: Authentication & Session Management / Rate Limiting

Affected Components: Unlock screen/modal, transaction-signing prompt, Midnight Nodes configuration

Description

The wallet currently allows unlimited consecutive password attempts when unlocking the extension, with no backoff, lockout, or escalation. This materially lowers the practical cost of online brute-force guessing. Even if KDF parameters are strong, the absence of throttling allows sustained guessing over time and increases exposure if users chose weak or reused passwords. There is also no user-facing feedback about failed-attempt counts or protections against automated retries.

Impacted flows: the password gate is required to open the wallet (unlock), sign transactions, and Configure Midnight Nodes. All of these surfaces currently allow unlimited consecutive attempts without backoff or lockout, inheriting the same brute-force risk.

Recommendation

- Introduce rate limiting & backoff: add exponential backoff per profile (e.g., 1s → 2s → 4s ... capped at 60s).
- Temporary lockout after N failures: e.g., 10 consecutive failures → 15–30 min lock; show remaining attempts and cooldown timer.
- Persist counters across restarts: store failure counters/cooldowns so relaunching the extension doesn't reset protections.
- Offer safe recovery path: "Forgot password" flow that requires seed re-import; never weaken KDF or bypass checks.
- Support stronger unlock factors (optional): OS biometrics/secure-enclave gating the decrypt step (without exporting secrets).
- Defend against DoS: allow the legitimate user to override lockout with seed-based rebind rather than waiting out cooldown.

[L 202] Password Policy Allows Weak Passwords

	Level	Severity	Status
	2	Low	Acknowledged

Category: Authentication & Session Management / Password Hardening

Affected Components: Password creation/change UI; strength meter

Description

The wallet's password screen accepts low-entropy passwords and visually indicates acceptability ($\approx 3/4$ green bars). Independent offline estimation with `zxcvbn` v4.4.2 shows that the accepted examples provide $1.0e8$ – $2.33e8$ estimated guesses of resistance (score 3), which is well below a conservative $1e10$ guesses bar typically used to screen weak choices.

In practice, this means users can set passwords that satisfy the meter's rules yet remain practically guessable, creating unnecessary exposure.

Evidence

- UI behavior: Meter displays $3/4$ green bars for:
 - `mnbkjhjhg` (lowercase keyboard-ish)
 - `1232343454` (numeric pattern)
 - `e3123qwea` (simple letters+numbers)
- Offline estimation (authoritative): `zxcvbn` v4.4.2, threshold = $1e10$ guesses.

1	password	score	guesses	guesses_log10	crack_time (hh:mm:ss)	verdict
2	-----	----	-----	-----	-----	-----
3	mnbkjhjhg	3	233,320,000	8.368	6:00:00	weak
4	1232343454	3	138,405,400	8.141	4:00:00	weak
5	e3123qwea	3	100,330,000	8.001	3:00:00	weak

Verdict rule: a password is weak if `score  $\leq$  2` or `estimated_guesses <  $1e10$` .

Note: Results are derived from `zxcvbn` v4.4.2; public web meters were not used as a decision gate. (crack time assumes 10k guesses/sec under a slow-hash model)

Impact

- Reduced guess resistance: Accepted passwords withstand only hours of offline guessing at conservative rates, making compromise feasible if password material is exposed (e.g., endpoint compromise, local profile theft, or backup leak).
- User risk amplification: The meter's green-bar presentation can mislead users into believing these choices are strong.
- Compounds other risks: In combination with other weaknesses (e.g., unlimited password attempts, tracked separately), overall account protection degrades.

Likelihood

Moderate. Users routinely choose memorable patterns; the current meter/validation permits such choices.

Reproduction (minimal)

1. Start wallet setup → password screen.
2. Enter `mnbkjhjhg`, `1232343454`, or `e3123qwea`.
3. Observe the meter (3/4 green bars).
4. Independently evaluate the same strings with `zxcvbn v4.4.2` (or an equivalent offline estimator) and compare estimated guesses to a `1e10` threshold.

Recommendations

- Gate on guess-resistance:
 - Reject when `zxcvbn.score ≤ 2` or `estimated_guesses < 1e10`.
 - Accept only if `score ≥ 3` and `guesses ≥ 1e10`.
 - Ensure the meter's visual state matches this gate.
- Block compromised/common values:
 - Screen against commonly used and breached passwords during creation/change.
- Passphrase support & input hygiene:
 - Length: min 12–15, max 128 (no silent truncation; show remaining chars).
 - Characters: allow spaces & Unicode; reject control/non-printing; if leading/trailing spaces are present, warn but allow.
 - Normalization: apply NFC consistently on set/verify (or store/compare consistently).
 - Strength gate: breach check + `zxcvbn` (e.g., `guesses ≥ 1e10` or `score > 2`).
 - Usability: allow paste/password managers; avoid composition rules (no forced symbol/case mix).
- Meter UX: ensure visual bars/labels align with the gate (don't show "acceptable" for values that would be rejected).
- Password Generator (Optional):
 - Default to ≥16–20 characters with sufficient entropy; avoid predictable templates (e.g., `<word><year>!`).
 - Verify generator outputs also satisfy the above recommendations.

[L 203] Feature-flag cache is controllable via page localStorage

	Level	Severity	Status
	2	Low	Acknowledged

Category: Extension And Storage Security

Description

During testing, any website that loads the Lace content scripts was able to read and overwrite the `lace-features` entry in its own `window.localStorage`. The extension checks this page-owned value first each time the content script initializes. If the key exists, the cached JSON is returned immediately and used to decide which modules run on that origin, while the service worker refresh happens asynchronously in the background. A rogue origin can therefore pre-populate `lace-features` with crafted data before Lace initializes.

Because feature flags specify the target blockchain network (mainnet, testnet, etc.), the Midnight proof server, and potentially security-sensitive modules, a hostile site can modify these flags without needing the user to interact. The attack inherits the extension's global `<all_urls>` injection, meaning any site—including compromised ad frames or phishing portals—can prime the cache and race the legitimate service worker response. This violates the trust boundary that should exist between untrusted web storage and privileged extension code, effectively giving arbitrary origins limited control over the wallet's runtime feature set.

Recommendation

- Avoid using page `localStorage` as the authoritative cache for feature flags. Persist flags inside extension-controlled storage (e.g., `chrome.storage.local` or an in-memory background cache) and provide read-only snapshots to content scripts via messaging so untrusted origins cannot forge state.
- If a local cache is required for performance, cryptographically sign or MAC the serialized payload and drop any entry that fails verification before use.

[L 204] Wallets never auto-lock after unlock

	Level	Severity	Status
	2	Low	Acknowledged

Category: Authentication & Session Management

Description:

During testing, unlocking a Midnight wallet and leaving Lace idle for several minutes revealed that the wallet doesn't eventually re-prompt for the password. Closing the popup, refreshing the extension, or even waiting for Chrome to suspend/resume the background service worker still kept the wallet fully unlocked. Only closing the browser completely or clearing the cache seemed to force re-authentication with the password. No idle timer, absolute session timeout, or "lock after X minutes" control was observed anywhere in the UI.

Recommendation:

- Implement both idle (e.g., 5–10 minutes of inactivity) and absolute (e.g., 8–12 hours) timers that automatically re-lock the wallet and clear in-memory secrets, even if the popup is closed.
- Hook into Chrome/Firefox `idle` APIs plus service-worker suspend/resume events so the timers fire even if the popup closes or the background script restarts.
- Consider a user-facing indicator and optional "lock now" control so users can confirm when the Midnight wallet has been re-secured.

[L 205] Missing CDDL/Bech32 Address Validation

	Level	Severity	Status
	2	Low	Acknowledged

Category: User Experience & Education

Affected Files:

- `packages/contract/midnight-context/src/address-util.ts`
- `packages/module/blockchain-midnight/src/exported-modules/address-validator.ts`

Affected Components: Address validation, Send flow, Transaction confirmation

Description

Address validation relies solely on regular expressions without proper CDDL schema or Bech32 checksum verification. The Midnight SDK provides codec-based validation (`ShieldedAddress.codec.decode`, `UnshieldedAddress.codec.decode`), but the current implementation uses pattern matching only.

1. Regex-only validation (no Bech32 checksum)

- Observed: `packages/contract/midnight-context/src/address-util.ts` matches addresses by regex only:
 - Shielded: `/^mn_shield-addr(?:_(?:test|dev|undeployed))?1(?:[qpzry9x8gf2tvdw0s3jn54khce6mua7l]{80,150})$/`
 - Unshielded: `/^mn_addr(?:_(?:test|dev|undeployed))?1[02-9ac-hj-np-z]{58}$/`
- Why problematic: Bech32 encodes a checksum that detects common typos (1–4 char errors). Regex cannot verify this, so a mistyped address like `mn_addr_test1...` with one wrong character may pass validation, potentially resulting in unrecoverable fund loss.

2. Unsafe boundary: send flow relies on regex helpers

- Observed: `packages/module/blockchain-midnight/src/exported-modules/address-validator.ts` calls `isValidMidnightAddress(address)` which only consults the regex helpers above; codecs are not invoked at validation time.
- Why problematic: Inputs from clipboard/typing are treated as valid if they match the regex, without checksum and network-ID checks. This creates potential exposure to clipboard-hijacking and phishing attempts that introduce subtle, invalid addresses.

These issues don't directly enable an exploit by themselves but increase the risk of user-induced loss (typos, clipboard tampering) during sending.

Code evidence:

```
1 // packages/contract/midnight-context/src/address-util.ts typescript
2 export const isShieldedAddress = (address: string) =>
```

```

3  /^mn_shield-addr(?:_(?:test|dev|undeployed))?1(?:[qpzry9x8gf2tvdw0s3jn54khce6mua7l]
   {80,150})$/ .test(address);
4
5  export const isUnshieldedAddress = (address: string) =>
6  /^mn_addr(?:_(?:test|dev|undeployed))?1[02-9ac-hj-np-z]{58}$/ .test(address);
7
8  // packages/module/blockchain-midnight/src/exported-modules/address-validator.ts
9  validateAddress: (address: string, token?: Token) => {
10   if (!token)
11     return isValidMidnightAddress(address) // regex only; no codec decode
12     ? None
13     : Some(new Error(errorMessage.invalidAddress));
14 }

```

Recommendation

- Switch to SDK codec validation
 - Use `ShieldedAddress.codec.decode()` and `UnshieldedAddress.codec.decode()` to validate structure and checksum.
- Enforce network ID matching
 - Verify testnet/mainnet tags and reject cross-network addresses with specific error messages.
- Improve UX error messages
 - Distinguish between checksum failures (likely typos), invalid format, and network mismatch; encourage re-checking clipboard.
- Add unit and e2e tests
 - Cover checksum failure cases, near-miss typos, and cross-network scenarios; include property tests for decoder robustness.

[I 101] Unsafe Onboarding Flows

	Level	Severity	Status
	1	Informational	Communicated

Category: User Experience & Education / Onboarding Safety

Affected Components: Onboarding flow, seed/secret phrase screens, confirmation step, clipboard feature, password creation screen, keyboard navigation/tab order

Description

The current onboarding emphasizes speed but does not require users to demonstrate understanding of critical safety topics before completing setup. Specific gaps include:

1. Seed phrase shown immediately

- Observed: Tapping “New wallet (Create a new Lace wallet)” displays the seed phrase directly.
- Why problematic: Exposes highly sensitive material without deliberate user action; increases shoulder-surfing/screenshot risk and bypasses an explicit safety reminder.

2. Confirmation step requires pasting the full seed

- Observed: Confirming the secret recovery phrase asks the user to paste the entire phrase.
- Why problematic: Encourages keeping the phrase in digital form and trains unsafe behavior. Proper confirmation should verify comprehension (ordered recall of a few random words), not create a copyable digital artifact.

3. One-click “Copy to clipboard” for the seed

- Observed: A button copies the entire seed to the system clipboard. A hover message says “Before using this feature, please read this best practices FAQ,” but appears only on hover.
- Why problematic: Any malware or app with clipboard access could exfiltrate the seed. The current hint is easy to miss; there’s no explicit opt-in, modal disclaimer, or guidance to clear the clipboard.

4. Password immutability not clearly communicated

- Observed: The wallet name/password page does not clearly warn that the password cannot be changed later and cannot be recovered by Lace.
- Why problematic: Users may choose weak or disposable passwords under the false impression they can change or recover them later.

5. Password field tab order can accidentally reveal the password

- Observed:
 - After entering the password, pressing `Tab` focuses the show/hide password toggle instead of the “re-type password” field. If the password includes spaces (which are allowed) and the user, by muscle memory, starts re-typing the password, pressing `Space` will toggle visibility and reveal the password on-screen.

- After confirming the password, `Tab` again moves focus back to the show/hide control instead of proceeding directly to the next logical step.
- Why problematic:
 - Unexpected focus behavior combined with `Space` toggling visibility makes accidental password disclosure more likely, especially in shared or recorded environments. Users may not notice that they revealed the password while assuming they are typing into a masked field.
 - It also degrades keyboard-only user experience: the user must press `Tab` twice instead of once to reach the confirmation field, and after confirmation the flow cycles through the show/hide toggle again instead of moving directly to the `Next` control.

6. Seed phrase remains visible while the educational video is open

- Observed: On the “Recovery phrase” page, the text says: “Consider your recovery phrase as the master key to your wallet. Watch video.” Clicking “Watch video” opens a popup that hides most of the seed phrase, but a few words remain visible. If the user opens the video in a new tab, the original tab continues to expose the seed phrase.
- Why problematic:
 - During the video, parts of the seed remain exposed. This keeps highly sensitive information visible longer than necessary.
 - It also mixes education with live secret exposure: the seed-display screen is not the right moment to educate. Once the recovery phrase is generated and visible, the UX should focus solely on secure backup and minimizing exposure, not introducing additional content.

These issues don’t constitute a direct exploit but meaningfully increase the likelihood of user-induced loss (e.g., poor backups/lost access, phishing approvals, clipboard exfiltration, and screenshot/shoulder-surfing leaks) at first run.

Recommendation

1. Gate seed reveal behind deliberate action and warning

- Default to blurred/hidden seed with a “Tap to reveal” affordance.
- On tap, show a modal disclaimer (e.g., “Make sure no one is watching your screen. Never share this phrase. Anyone with it can access your wallet.”) with Continue to reveal / Cancel.
- Consider auto-hide/blur after inactivity and require re-reveal.

2. Replace full-paste confirmation with a randomized quiz

- Ask for 3 randomly selected words (positions vary per attempt).
- Re-randomize on failure and enforce a sensible attempt limit / short cooldown.
- Do not encourage storing the full seed digitally; avoid paste-from-clipboard UX here.

3. Make clipboard copy an explicit, gated opt-in

- Require ticking an “Enable copy” checkbox that triggers a modal risk explanation (malware/clipboard access, best practices, preference for offline paper/metal backup).
- After copying, show clear-clipboard guidance (and where platform-allowed, offer a “Clear clipboard now” action).
- Keep the warning visible on click, not only on hover.

4. Communicate password immutability and recovery limits up front

- Add a clear notice: “This password cannot be changed later and cannot be recovered by Lace.”
- Encourage password manager use before proceeding.
- Require an explicit acknowledgment checkbox before continuing.

5. Fix keyboard focus order and toggle behavior on the password form

- Ensure the `Tab` order moves from the password field directly to the “re-type password” field, not to the show/hide toggle.
- Treat the show/hide control as non-focusable by default in the primary flow, or place it outside the main logical tab order so it does not sit between the password, confirmation, and next-step controls.

6. Hide the seed phrase fully while the educational video is active

- When “Watch video” is opened (whether in a popup or new tab), fully hide or blur the seed phrase on the current page.
- Restore visibility only when the user explicitly returns to the seed screen and re-confirms intent to reveal (e.g., requiring another “Tap to reveal”).
- Ensure no seed words remain visible in any overlay, partial scroll, or background state while the video is playing.
- Alternatively, place the “Watch video” call-to-action on a previous, purely educational screen where no recovery phrase has been generated or revealed yet, so users can view the content without any secret material present.

Resolution

Communicated to client; no explicit remediation decision at the time of writing.

[I 102] Missing **Contacts / Address Book** Feature

	Level	Severity	Status
	1	Informational	Communicated

Category: User Experience & Education / Safety Nets

Affected Components: Send flow (address input), recent recipients, transaction review UI

Description

The wallet lacks a built-in **Contacts / Address Book** feature. Users must repeatedly copy-paste long bech32 addresses or scan ad-hoc QRs, which is error-prone and increases the risk of typos, clipboard tampering, or wrong-recipient sends. Without first-class saved contacts (with labels, notes, and network/asset constraints), the UI cannot warn when an entered address doesn't match a known contact, nor can it surface recipient history, block-listed addresses, or per-contact safety hints (e.g., "testnet-only", "DAO treasury", "exchange deposit").

These gaps don't constitute a direct exploit, but they significantly increase the likelihood of user-induced loss (misaddressed transactions, phishing-induced pastes, clipboard exfiltration) and heighten exposure to address-poisoning scams, both at first run and during routine use.

Recommendation

- **Add a **Contacts / Address Book**:**
 - Create, edit, and delete contacts with name, address, optional notes, and network scope (mainnet/testnet).
 - Persist recent recipients with a one-tap "Save as Contact."
 - Support import/export (encrypted), and watch-only tags.
- **Inline safety integrations:**
 - When pasting/typing, suggest matching contacts and show clear identity cues (avatar/label).
 - Warn if the target isn't a saved contact or if it conflicts with the contact's network/asset constraints.
 - Offer blocklist and "mark as risky" flags; surface these in the tx review step.
- **Quality-of-life & anti-error features:**
 - QR code save/share for contacts; optional nicknames in activity.
 - (Optional) Domain/resolution integration (future-proof) with clear, verifiable provenance.

Resolution

Communicated to client; no explicit remediation decision at the time of writing.

[I 103] Unimplemented Wallet API Codecs

	Level	Severity	Status
	1	Informational	Communicated

Category: User Experience & Education

Affected Files:

- `packages/module/dapp-connector-midnight/src/codec/decodeWalletApi.ts`
- `packages/module/dapp-connector-midnight/src/codec/encodeWalletApi.ts`

Affected Components: Wallet API codecs, transaction validation, dApp connector

Description

Wallet API codec functions are unimplemented stubs that perform type casting without any runtime validation. This creates risk when transaction data crosses boundaries such as Chrome extension messaging, storage, or inter-process communication.

1. Type casting instead of actual codec implementation

- Observed: `packages/module/dapp-connector-midnight/src/codec/decodeWalletApi.ts` and `encodeWalletApi.ts` return empty objects or use type assertions (`tx as unknown as Transaction`) with TODO comments indicating they need proper implementation.
- Why problematic: When transaction data is serialized/deserialized across extension messaging boundaries or stored/retrieved from persistence, there's no validation that the data structure is valid. Invalid or corrupted transaction data won't be detected until it causes runtime errors during transaction execution. This may result in wallet crashes, failed transactions, or inconsistent state.

2. Unsafe type assertions (`as unknown as`) at the wallet API boundary

- Observed: Codec stubs and API paths return typed values without decoding/validation, relying on casts:
 - `src/codec/decodeWalletApi.ts`: `return {} as ConnectedAPI;`
 - `src/codec/encodeWalletApi.ts`:
`return {} as WithSenderContext<SerializedDAppConnectorWalletAPI>;`
 - `submitTransaction(tx)`: `tx as unknown as TransactionType`
- Why problematic: `as unknown as ...` bypasses all runtime checks. Messages crossing extension/storage boundaries can be malformed or version-mismatched; without schema-backed decoding at the boundary, bad payloads may propagate and fail deeper (crashes/inconsistent state) instead of being rejected early with a clear error.

These issues do not constitute a direct exploit while the dApp connector is disabled, but they may increase the chance of silent parsing/validation failures once enabled, making transaction-manipulation bugs harder to detect and recover from.

Code evidence:

```

1 // src/codec/decodeWalletApi.ts
2 export const decodeWalletApi = (
3   _walletApi: SerializedDAppConnectorWalletAPI,
4 ): ConnectedAPI => {
5   return {} as ConnectedAPI; // type assertion, no decoding
6 };
7
8 // src/codec/encodeWalletApi.ts
9 export const encodeWalletApi = (
10  _walletApi: WithSenderContext<ConnectedAPI>,
11  _getNetworkId: () => Promise<SupportedNetworkId>,
12 ): WithSenderContext<SerializedDAppConnectorWalletAPI> => {
13   return {} as WithSenderContext<SerializedDAppConnectorWalletAPI>; // type assertion,
    no encoding
14 };
15
16 // submitTransaction path
17 await wallet.submitTransaction(tx as unknown as TransactionType); // unsafe cast

```

Recommendation

1. Implement schema-backed codecs
 - Use CDDL/Compact schemas and SDK-provided types for all wallet API request/response shapes.
2. Validate at trust boundaries
 - Enforce decoding/encoding at extension messaging, persistence, and network boundaries; reject malformed payloads with explicit errors.
3. Remove unsafe type assertions
 - Replace `as unknown as Type` patterns with proper codec parse/constructors; propagate typed results.
4. Add robust tests
 - Include property/fuzz tests for codecs, round-trip tests, and regression tests for edge cases/version mismatches.
5. Introduce versioning and error handling
 - Embed version fields in payloads; provide clear, user-safe error messages when decoding fails.

Resolution

Communicated to client; no explicit remediation decision at the time of writing.

[I 104] Suggestion: TOR Network Anonymity as Optional Privacy Enhancement

	Level	Severity	Status
	1	Informational	Communicated

Category: Network Privacy & User Protection

Affected Components: Network communication, blockchain API providers, user privacy metadata

Type: Privacy Enhancement Suggestion

Background: What is TOR (The Onion Router)?

TOR is a free, open-source anonymity network that protects your privacy by hiding your IP address and location when using the internet. Think of it as routing your internet traffic through multiple encrypted layers (like an onion) so that no one can trace your online activity back to you.

How TOR Works

1	Your Computer → Entry Node → Middle Node → Exit Node → Destination	text
2	[Layer 3 encryption]	
3	[Layer 2 encryption]	
4	[Layer 1 encryption]	
5	[Decrypted]	

The “Onion” Concept:

- Your data is wrapped in multiple layers of encryption (like layers of an onion)
- Each node in the TOR network “peels away” one layer of encryption
- No single node knows both your identity AND your destination
- Your IP address is hidden from the websites you visit

TOR and Blockchain Privacy:

Standard network connections:

- Blockchain API providers can observe your IP address
- Your ISP can observe cryptocurrency usage
- Network observers may correlate transactions with locations
- Provider logs may contain user metadata

With TOR enabled:

- Your IP address is obscured by TOR exit nodes
- API providers observe TOR exit nodes rather than user IPs
- Network metadata (timing, location) is more difficult to correlate
- Additional privacy layer spanning network and application levels

Summary

This suggestion explores implementing TOR (The Onion Router) network anonymity support in Lace Midnight wallet as an optional privacy enhancement. Midnight provides on-chain privacy through zero-knowledge proofs. TOR integration would add an additional layer of network-level anonymity for users who require IP address protection.

Potential Benefits of TOR Integration:

- Layered Privacy Approach: Combines on-chain privacy (ZK-proofs) with optional network-layer anonymity
- Industry Alignment: Available in some privacy-focused wallets (Wasabi, Samurai, Monero)
- Optional User Protection: Reduces risk of IP-based correlation for users who enable it
- Feature Option: Provides network-layer privacy comparable to other privacy-focused blockchain wallets

Network Privacy Considerations

Network metadata in current implementation

- Current State: All network requests to blockchain providers (Blockfrost for Cardano, Maestro for Bitcoin, Midnight RPC endpoints) are made directly from user's IP address without any anonymity layer. Network traffic analysis shows:
 - `packages/module/cardano-provider-blockfrost/src/blockfrost/blockfrost-provider.ts`: Direct HTTP client to Blockfrost API
 - `packages/module/bitcoin-provider-maestro/src/maestro/maestro-provider.ts`: Direct HTTP client to Maestro API
 - `packages/module/blockchain-midnight/src/store/dependencies.ts`: Direct connections to Midnight network nodes
 - No TOR/proxy/anonymity configuration options in codebase
- Network Layer: Midnight uses zero-knowledge proofs to hide transaction details on-chain (amounts, participants). At the network layer, the following metadata is visible:
 - IP Address Correlation: API providers can correlate wallet addresses with user IP addresses and geolocations
 - Transaction Timing: Network observers can link transaction timing patterns to specific users
 - Metadata Leakage: ISPs, VPN providers, and API servers log all blockchain interactions with source IPs
 - Cross-Chain Analysis: Since wallet supports multiple blockchains, metadata from one chain can compromise privacy on others

Privacy Implications & Use Cases

Use Case 1: Privacy-Conscious Users

1	User creates shielded Midnight transaction (on-chain private via ZK-proofs)	text
2	↓	
3	WITHOUT TOR: Midnight RPC logs IP address (123.45.67.89, NYC, Verizon ISP)	
4	↓	
5	WITH TOR: Midnight RPC only sees TOR exit node IP (Unknown user location)	
6	↓	
7	Result: Enhanced privacy - on-chain privacy combined with network-layer anonymity	

Use Case 2: Cross-Chain Privacy Protection

1	User holds tokens on multiple chains (Cardano ADA, Midnight NIGHT, Bitcoin BTC)	text
2	↓	
3	WITHOUT TOR: All API providers observe same user IP (123.45.67.89)	
4	→ Analysts may correlate: Cardano, Midnight, Bitcoin addresses to same user	
5	↓	
6	WITH TOR: Each request uses different TOR circuit/exit node	
7	→ Cross-chain activity correlation becomes more difficult	

Use Case 3: Network Privacy in Various Jurisdictions

1	User in jurisdiction with cryptocurrency monitoring	text
2	↓	
3	WITHOUT TOR: ISP may observe user accessing blockchain networks	
4	→ Blockchain usage is visible at network level	
5	↓	
6	WITH TOR: ISP observes encrypted TOR traffic	
7	→ Specific blockchain activity is obscured at network level	

Trade-offs and Limitations of TOR Integration

Performance Impact:

- TOR routing adds latency (typically 2-5x slower than direct connections)
- May impact user experience for time-sensitive operations
- Increased timeout requirements for network requests

Complexity Considerations:

- Additional dependency management (TOR daemon or proxy libraries)
- Increased attack surface (TOR vulnerabilities, exit node risks)
- Browser extension limitations may restrict native TOR integration

User Experience:

- Requires user education about TOR benefits and trade-offs

- Optional feature means many users may not enable it
- Connection reliability may vary based on TOR network health

Security Considerations:

- TOR exit nodes can observe unencrypted traffic (mitigated by HTTPS)
- Some blockchain API providers may block TOR exit nodes
- Potential timing attacks on TOR network (though difficult to execute)

Implementation Overhead:

- Development and maintenance costs for TOR integration
- Testing complexity across different platforms and configurations
- Support burden for troubleshooting TOR-related issues

Comparison with industry practices

TOR implementation in other privacy-focused wallets

- Wasabi Wallet (Bitcoin): TOR integration by default, routes all traffic through TOR network
- Samourai Wallet (Bitcoin): Built-in TOR support, Whirlpool coinjoin over TOR
- Monero GUI Wallet: Native TOR integration for transaction broadcast
- Sparrow Wallet (Bitcoin): TOR/proxy support for all network communication

Academic Research Context

- “Bitcoin over Tor isn’t a good idea” (Biryukov & Pustogarov, 2015): Discusses network-level metadata exposure in Bitcoin
- “Deanonymizing Tor hidden service users through Bitcoin transactions analysis” (2018): Explores blockchain metadata correlation techniques
- TOR Project blog (2019): Discusses privacy considerations for cryptocurrency users

Considerations for TOR Integration

Current Privacy Features

Midnight already provides strong privacy protections:

- Zero-knowledge proofs: Hide transaction amounts, participants, and details on-chain
- Shielded transactions: Protect transaction privacy at the application layer
- EMIP-3 encryption: Secures private keys and sensitive data in storage

TOR integration would complement these existing features by adding network-layer anonymity.

Layered Privacy Architecture

- Application Layer: Zero-knowledge proofs hide transaction details (currently implemented)
- Network Layer (with TOR): IP addresses remain anonymous (optional enhancement)
- Result: Privacy across both network and application layers

User Privacy Considerations

- Users selecting Midnight may have strong privacy preferences
- Network-layer privacy is available in some privacy-focused wallets
- Other privacy-focused blockchain wallets (Monero, Zcash) offer TOR support
- TOR integration would provide additional privacy options for users who require it

Technical Feasibility

- TOR integration patterns exist from cryptocurrency wallet implementations (10+ years of examples)
- Libraries available: `tor-request` (Node.js), SOCKS5 proxy support
- Can be implemented as optional feature with user consent
- Performance trade-off (slower speeds) may be acceptable for privacy-conscious users

Implementation Recommendations

1. TOR Integration Options

Option A: Native TOR Integration

- Embed TOR client or use system TOR daemon (similar to Wasabi Wallet approach)
- Route all blockchain API traffic through TOR SOCKS5 proxy
- Advantages: Strongest privacy, user doesn't need external setup
- Disadvantages: Larger bundle size, TOR daemon management complexity

Option B: External TOR Proxy Support

- Add SOCKS5/HTTP proxy configuration in wallet settings
- Users can point to their own TOR daemon (localhost:9050)
- Advantages: Lightweight, flexible, power users can control TOR setup
- Disadvantages: Requires user to install/configure TOR separately

Implementation Approach

```

1 // packages/lib/util-provider/src/http-client.ts
2 export const createHttpClient = (config: {
3   baseUrl: string;
4   torEnabled?: boolean;
5   proxyUrl?: string;
6 }) => {
7   const agent = config.torEnabled
8     ? new SocksProxyAgent('socks5://127.0.0.1:9050') // TOR SOCKS5 proxy
9     : undefined;
10
11   return axios.create({
12     baseUrl: config.baseUrl,
  
```

```
13     httpAgent: agent,  
14     httpsAgent: agent,  
15     timeout: config.torEnabled ? 30000 : 10000, // Longer timeout for TOR  
16   });  
17 };
```

UI/UX Considerations

- Add “Enable TOR Network Privacy” toggle in settings
- Display warning: “TOR network is slower but protects your IP address”
- Show TOR connection status indicator (connected/disconnected/error)
- Provide educational content explaining IP privacy risks

2. Implement Privacy-Preserving Alternatives

If full TOR integration is not immediately feasible:

Option C: Random RPC Endpoint Rotation

- Rotate between multiple independent API providers for each request
- Makes correlation harder (no single provider sees all activity)
- Disadvantages: Doesn’t hide IP, just distributes metadata across providers

Option D: VPN/Proxy Recommendations

- Document how users can route wallet traffic through VPN/TOR
- Provide setup guides for Tor Browser + extension usage
- Add in-app warnings about IP privacy risks
- Disadvantages: Relies on user education and external tools

3. Short-term Mitigations (Before TOR Integration)

Transparency about IP exposure

- Add privacy notice: “Your IP address is visible to blockchain API providers”
- Warn users that network metadata can compromise on-chain privacy
- Link to privacy best practices documentation

API provider selection

- Choose providers with strong privacy policies
- Require providers to commit to minimal logging
- Rotate API keys to prevent long-term user profiling

Documentation and user education

- Explain the difference between on-chain privacy (ZK-proofs) and network privacy (IP anonymity)
- Provide guides for using wallet with external TOR setup
- Recommend privacy-preserving network configurations

4. Long-term Consideration: Full-Stack Privacy

Complete Privacy Stack

1	Layer 1: Network Anonymity (TOR/I2P)	text
2	↓	
3	Layer 2: Transport Encryption (TLS/HTTPS)	
4	↓	
5	Layer 3: Application Privacy (ZK-Proofs)	
6	↓	
7	Layer 4: Storage Security (EMIP-3 encryption)	

Potential Outcomes

- May support privacy-focused user requirements
- Provides option for users concerned about metadata surveillance
- Feature differentiation from wallets without network-layer privacy
- May appeal to privacy-conscious users who require IP anonymity

Potential Timeline

- v1.0: Add proxy/SOCKS5 support + documentation
- v1.5: Native TOR integration (optional feature)
- v2.0: TOR enabled by default with opt-out option

References

Research Papers

- Biryukov & Pustogarov (2015): "Bitcoin over Tor isn't a good idea" - <https://arxiv.org/pdf/1410.6079>
- Fanti et al. (2018): "Deanonymizing Tor hidden service users through Bitcoin transactions analysis" - <https://arxiv.org/pdf/1801.07501>

Industry Examples

- Wasabi Wallet TOR integration: <https://docs.wasabiwallet.io/using-wasabi/NetworkLevelPrivacy.html>
- Samourai Wallet TOR documentation: <https://docs.samourai.io/en/wallet/tor-vpn>
- Tor Project cryptocurrency guide: <https://blog.torproject.org/tor-heart-cryptocurrencies/>

OWASP Privacy Guidance

- OWASP Top 10 Privacy Risks: <https://owasp.org/www-project-top-10-privacy-risks/>
- Privacy Risk 8: Lack of anonymization - Network identifiers enable user tracking

Resolution

Communicated to client; no explicit remediation decision at the time of writing.

End of Report



No Witness Labs — Confidential

Copyright © 2025